Vector Autograd, Initialisation and Valid Outputs

Deep Learning course notes Sources: listed under References

What this file is really about:

$$\text{scalars} \rightarrow \text{Jacobians} \rightarrow \text{vector-Jacobian products} \rightarrow \text{keep signals alive} \rightarrow \text{make invalid outputs}$$

How to use this file. Part A extends autograd to arrays. Part B shows the code. Part C is practice with answers.

Step-up path

You need. Scalar autodiff and matrix products.

1. Read the shape of a Jacobian.
2. Replace it with a vector-Jacobian product.
3. Understand minibatch noise.
4. Initialise so signals survive.
5. Transform outputs so they are always valid.

You can stop when you can check the shapes of any backward pass and pick an initialisation.

Contents

Part A Maths From the Ground Up

A1 Shapes and Jacobians

For $f : \mathbb{R}^n \to \mathbb{R}^m$ the **Jacobian** is $m \times n$, with entry $\partial f_i / \partial x_j$. A dense layer $z = Wx + b$ has Jacobian W with respect to x.

A2 Vector-Jacobian products

Backpropagation never builds a Jacobian. Each operation only computes a **vector-Jacobian product**:

$$\delta_{\text{in}} = J^\top \delta_{\text{out}}.$$

For a dense layer this is $W^\top \delta$. This is why reverse mode stays cheap: each step is a product with a vector, and never a full matrix.

A3 Batches

For a batch, losses add over examples, so gradients add too. The gradient has the same shape as its parameter, and frameworks handle the batch dimension automatically.

A4 Minibatch noise

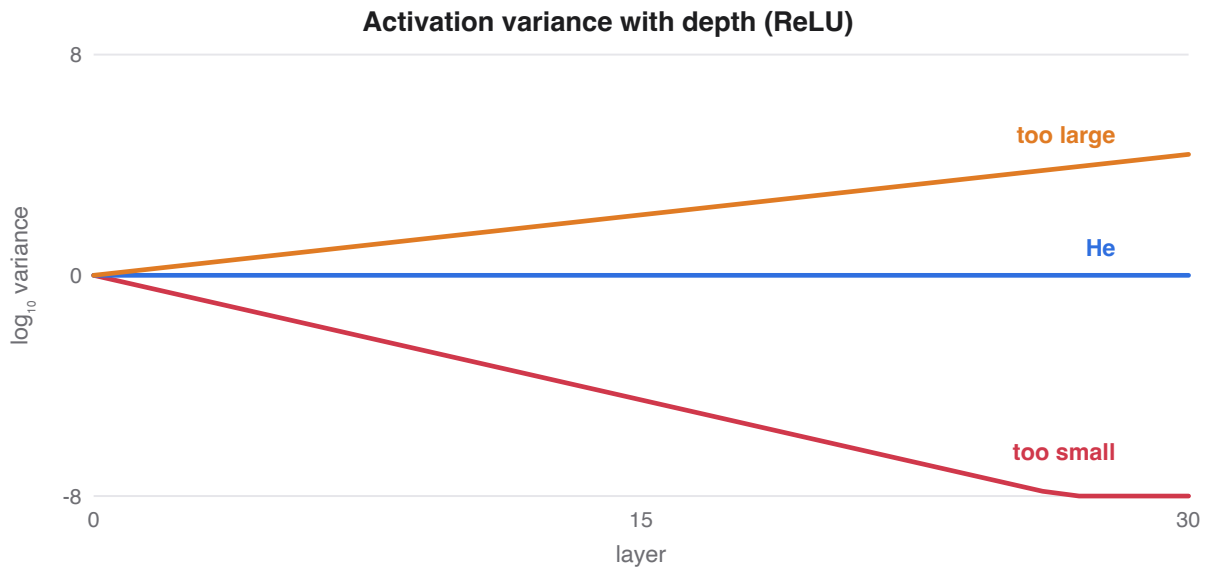
A minibatch gives an unbiased estimate of the full gradient. Its standard deviation falls like $1/\sqrt{b}$. A smaller batch means noisier steps, but many more of them for the same computation.

A5 Keeping signals alive

If each layer's weights are too small, activations shrink towards zero as depth grows. If too large, they blow up. With n inputs per unit, independent weights of variance σ_w^2 give

$$\operatorname{Var}(z) = n\,\sigma_w^2 \operatorname{Var}(x).$$

Xavier initialisation picks $\sigma_w^2 = 1/n$, which suits symmetric activations such as tanh. **He** initialisation picks $\sigma_w^2 = 2/n$, because ReLU zeroes half of its inputs.



A6 Make invalid outputs unreachable

If an output must be positive, or a set of outputs must add up to one, do not clip it or hope. Output an unconstrained number, and transform it:

Need	Transform
Positive	$\exp(u)$ or $\mathrm{softplus}(u) = \log(1 + e^u)$
Between 0 and 1	$\sigma(u)$
Probabilities	softmax
Between -1 and 1	$\tanh(u)$

A7 The gradient of a dense layer with softmax, derived

Take $z = Wx$ and $L = -\log \mathrm{softmax}(z)_y$. By the chain rule,

$$\frac{\partial L}{\partial z} = p - e_y,$$

where p is the softmax and e_y is the one-hot target. Since $z_i = \sum_j W_{ij} x_j$, we have $\partial z_i / \partial W_{ij} = x_j$, so

$$\nabla_W L = (p - e_y)\, x^\top.$$

The gradient is an outer product of output error and input. It has the same shape as W, and costs almost nothing to compute.

A8 Useful matrix derivatives

Function	Gradient
$a^\top x$	a
$x^\top A x$	$(A + A^\top)x$
$\|Ax - b\|^2$	$2A^\top(Ax - b)$
$a^\top W b$ with respect to W	$ab^\top$

A9 Why reverse mode costs about one forward pass

Each operation stores its inputs. On the way back, each performs one vector-Jacobian product, about as expensive as the original operation. So a full gradient, for millions of parameters, takes a small constant times the cost of one forward pass. Forward mode would need one pass per parameter.

A10 Deriving the initialisation rule

For $z = \sum_{i=1}^{n} w_i x_i$ with independent, zero-mean w_i and x_i,

$$\mathrm{Var}(w_i x_i) = \mathbb{E}[w_i^2]\mathbb{E}[x_i^2] = \sigma_w^2 \mathrm{Var}(x),$$

and independent variances add, giving $\mathrm{Var}(z) = n\sigma_w^2 \mathrm{Var}(x)$.

ReLU passes only positive values, and for symmetric z we have $\mathbb{E}[\mathrm{relu}(z)^2] = \frac{1}{2}\mathrm{Var}(z)$. To hold variance constant, we need $\frac{1}{2} n \sigma_w^2 = 1$, so $\sigma_w^2 = 2/n$.

Part B In Code

B1 Initialisation

```
nn.init.kaiming_normal_(layer.weight, nonlinearity='relu')
```

B2 A positive output

```
sigma = F.softplus(raw) + 1e-6
```

B3 Jacobians and vector-Jacobian products in practice

```
import torch
from torch.func import jacrev, vjp

W = torch.tensor([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])
f = lambda x: W @ x
x = torch.tensor([1.0, -1.0])

J = jacrev(f)(x)
assert J.shape == (3, 2) and (J == W).all()

_, pull = vjp(f, x)
delta = torch.tensor([1.0, 0.0, 2.0])
assert (pull(delta)[0] == W.T @ delta).all()
print('Jacobian shape', J.shape, 'vjp', pull(delta)[0])
```

B4 Verify the softmax gradient

```
import torch
from torch.nn import functional as F

W = torch.randn(3, 4, requires_grad=True); x = torch.randn(4)
y = 1
F.cross_entropy((W @ x).unsqueeze(0), torch.tensor([y])).backward()
p = F.softmax(W.detach() @ x, 0)
e = torch.zeros(3); e[y] = 1
assert (W.grad - torch.outer(p - e, x)).abs().max() < 1e-5
print('matches (p - e) x^T')
```

B5 Activation statistics across depth

Run this to see signals die, or blow up, under a poor initialisation.

```
import torch

torch.manual_seed(0)
def spread(gain, depth=20, n=512, act=torch.relu):
    x = torch.randn(2048, n)
    out = []
    for _ in range(depth):
        w = torch.randn(n, n) * gain / n ** 0.5
        x = act(x @ w)
        out.append(x.std().item())
    return out

print('too small', round(spread(0.5)[-1], 8))
print('He       ', round(spread(2 ** 0.5)[-1], 3))
print('too large', round(spread(3.0)[-1], 1))
assert spread(0.5)[-1] < 1e-4 < 0.1 < spread(2 ** 0.5)[-1] < 10 < spread(3.0)[-1]
```

Part C Practice

P1. $f : \mathbb{R}^3 \to \mathbb{R}^2$. What shape is the Jacobian?

2×3.

P2. How much noisier is a batch of 16 than a batch of 64?

Twice as noisy, since $\sqrt{64/16} = 2$.

P3. A ReLU unit has 100 inputs. Find the He variance of its weights.

$2/100 = 0.02$.

P4. Which transform gives a valid standard deviation?

Softplus or the exponential, since both are always positive.

P5. Find $\nabla_w \|Xw - y\|^2$.

$2X^\top(Xw - y)$.

P6. A ReLU layer has 200 inputs. What weight variance does He initialisation use?

$2/200 = 0.01$.